

Offline Field Collection Security Algorithm

Formal security and control specification



Document purpose

Define the controls that prevent an agent from silently resetting the MICROFI mobile application and continuing field collection after deleting or reinstalling the app.

Contents

1	Objective and Security Goal	3
2	Scope and Business Rules	3
3	Identity and Trust Model	4
4	Initial Enrollment Algorithm	4
5	Collection Record and Hash-Chain Model	5
6	Normal Online and Offline Collection	6
7	Server Synchronization and Verification Rules	6
8	Reinstallation / New Installation Control	7
9	Fraud Scenario: Offline Collections Then Uninstall/Reinstall	9
10	Administrator-Only Reset / Replacement	10
11	Daily Collection Ceiling (Escrow Limit)	10
12	Device Binding and Cryptographic Protections	11
13	Security States	12
14	Reference Pseudocode	13
15	Security Boundary and Limitations	14
16	End-to-End Algorithm Summary	15
17	Implementation Checklist	15
18	Technical References	16

■ Green marks accepted or normal outcomes.

■ Amber marks pending, offline or at-risk conditions.

■ Red marks rejected, blocked or security-event outcomes.

1 Objective and Security Goal

MICROFI allows field agents to perform financial collections even when Internet connectivity is unavailable. This capability creates a specific fraud risk: an agent may collect money offline, delete the application before synchronization, reinstall it, and attempt to continue collecting with an empty local history.

The system does not attempt to make application data impossible to erase. Android removes app-specific storage when an application is uninstalled, so local storage cannot be treated as the ultimate source of financial truth. The security goal is instead to ensure that a fresh installation does not automatically regain collection privileges when the server already has a trusted agent/device/installation state.

Security objective

A local reset must be non-authorizing. Only the administrator can authorize a new installation or device binding for an agent.

2 Scope and Business Rules

- ◆ Each collection agent is bound to **one authorized device** and **one authorized application installation** at a time.
- ◆ The first collection of a newly activated agent is **mandatory ONLINE**.
- ◆ After the first online collection, subsequent collections may be performed **ONLINE** or **OFFLINE**.
- ◆ Every collection has a monotonically increasing collection counter and a cryptographic linkage to the preceding collection.
- ◆ Each agent has a daily collection ceiling (escrow limit) configured when the agent is created. The ceiling is the maximum amount the agent may collect during one business day.
- ◆ A new installation is not automatically authorized. The agent cannot reset, replace, or rebind the installation himself.
- ◆ Any new installation or device presented for an already-bound agent creates a security event and blocks collection until an administrator explicitly resolves it.
- ◆ Administrative reset/replacement is an exceptional privileged operation and must be auditable.

3 Identity and Trust Model

The design deliberately separates the concepts of agent, device, installation, and collection state.

Entity	Meaning	Example	Server role
Agent	Business identity authorized to collect.	AG-1024	Owns authorization and limits.
Device	The authorized physical Android device identity established by a secure device-binding mechanism.	DEV-7A91	Binds financial operations to the approved device.
Installation	A specific installation instance of the MICROFI app. It may change after reinstall.	INS-4F22	Lets the server detect that a new installation is attempting to operate.
Collection state	Server-known counter, latest accepted hash/checkpoint, daily ceiling state and security status.	counter=153	Prevents a clean reinstall from being treated as a clean financial state.

Important

The Installation ID is not expected to survive uninstall/reinstall. The server must therefore not rely on Installation ID alone as proof of device continuity. The device-binding mechanism should use a cryptographic key protected by Android Keystore and a server-verifiable integrity/attestation signal. Android provides key-attestation capabilities, and Play Integrity provides server-verifiable application/device integrity signals. These mechanisms are complements, not substitutes for the server-side authorization state.

4 Initial Enrollment Algorithm

A newly created agent starts in an online-only onboarding state.

- 1 Administrator activates agent**
 Agent account is created/activated and the daily collection ceiling is configured.
- 2 First login**
 Agent authenticates to MICROFI over the network.
- 3 Create installation**
 The app creates a fresh Installation ID and generates the required cryptographic key material.

- 4

Device binding
The server validates the device-binding/attestation information and binds the authorized device and installation to the agent.
- 5

Offline remains disabled
The server marks the agent as `ONLINE_FIRST_COLLECTION_REQUIRED`.
- 6

First collection online
The agent performs Collection C1 while connected to the server.
- 7

Initialize counter/checkpoint
The server establishes the first trusted collection counter and ledger checkpoint.
- 8

Enable offline mode
Only after successful C1 acknowledgement does the server permit subsequent offline collection.

5 Collection Record and Hash-Chain Model

Each collection is represented by a signed record. At minimum, the record contains:

Collection record			minimum required fields
collectionId	agentId	authorizedInstallationId	deviceBindingReference
collectionCounter	businessDate	timestamp	client/account reference
amount	previousHash	currentHash	signature
syncStatus			

The chain is formed as follows:

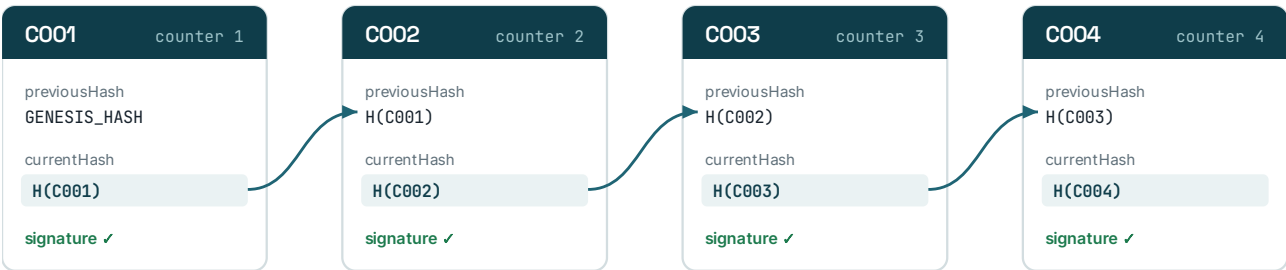


Figure 1 Each record's previousHash equals the currentHash of the record before it; the first record links to a genesis value.

For collection C002, previousHash equals the hash of C001; for C003, previousHash equals the hash of C002, and so on. The current hash is calculated over canonicalized collection data, the counter, the previous hash, and the relevant authorization context.

Tamper-evident is not enough

A hash chain by itself is only tamper-evident. To make the record attributable to the authorized installation, each collection should also be authenticated using a key whose private material is protected by Android Keystore where supported. The server verifies the signature before accepting the record.

6 Normal Online and Offline Collection

6.1 Online collection

- 1 Verify that the agent and installation are currently authorized.
- 2 Verify the agent daily ceiling and current business-date state.
- 3 Increment the collection counter.
- 4 Create the collection record, previousHash and currentHash.
- 5 Sign the canonical record using the installation/device signing key.
- 6 Send the collection to the server.
- 7 Server verifies authorization, signature, counter sequence, hash linkage, idempotency and daily ceiling.
- 8 Server acknowledges the collection and updates the trusted checkpoint.

6.2 Offline collection

- 1 Verify locally that the installation is still in a locally authorized state.
- 2 Verify the daily collection ceiling before accepting the new amount.
- 3 Increment the local collection counter.
- 4 Create the collection record and hash-chain link.
- 5 Sign the record using the local signing key.
- 6 Store the record in the encrypted local ledger/database with sync status **PENDING**.
- 7 When connectivity returns, synchronize pending records in counter order.

7 Server Synchronization and Verification Rules

Synchronization is a security boundary. The server must not merely accept whatever records arrive; it must verify continuity and reject ambiguous sequences.

Condition	Server action	Security meaning
Expected next counter received	Accept Accept after signature/hash/idempotency checks; update checkpoint.	Normal synchronization.

Condition	Server action	Security meaning
Duplicate already accepted	Return original ack Return the original acknowledgement; do not create a second financial record.	Protects against replay and retry.
Counter gap	Hold for review Do not silently continue. Mark synchronization as incomplete/suspicious and require missing records or administrative review.	A missing local segment may indicate data loss or tampering.
Unexpected older counter	Reject Reject unless it matches an idempotent previously accepted record.	Prevents rollback/reordering.
Invalid previousHash	Reject + security event Reject the record and create a security event.	Chain continuity has been broken.
Invalid signature	Reject + security event Reject the record and create a security event.	The record cannot be attributed to the authorized key.
Installation/device not authorized	Reject Reject financial operation.	A new or unauthorized installation cannot collect.
Daily ceiling exceeded	Reject Reject the collection.	Local or server-side limit violation.

8 Reinstallation / New Installation Control

The system does not claim to receive a direct “uninstall” message from Android. Instead, it detects that an already-authorized agent is presenting a new installation identity. This is sufficient because the business policy bans agent-initiated reinstall, phone replacement and factory reset for collection operations.

8.1 Login decision

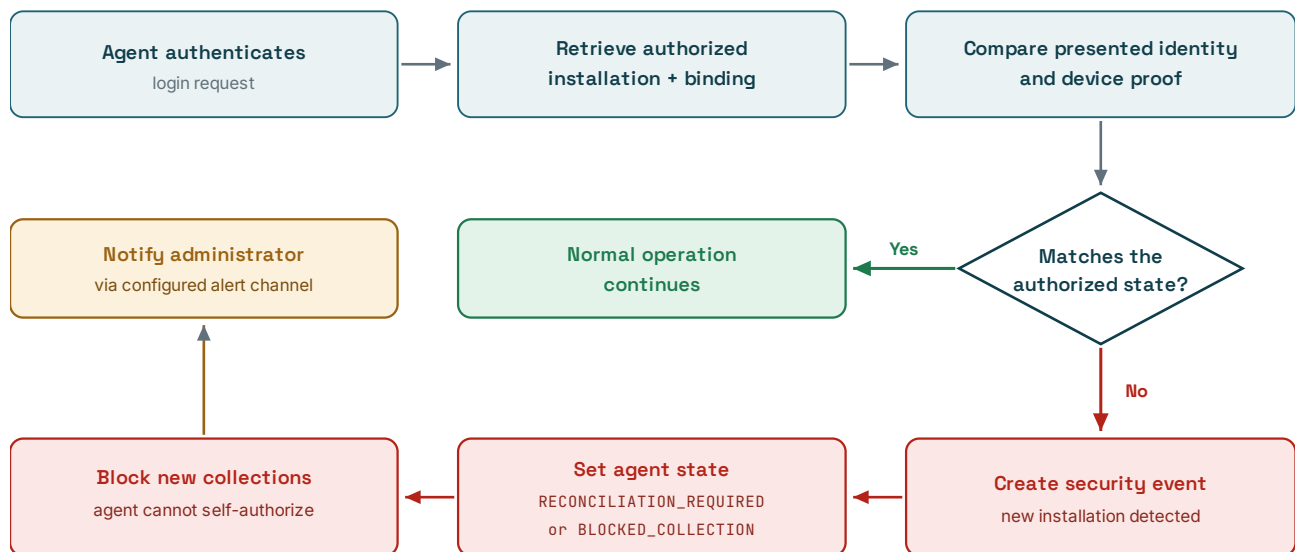
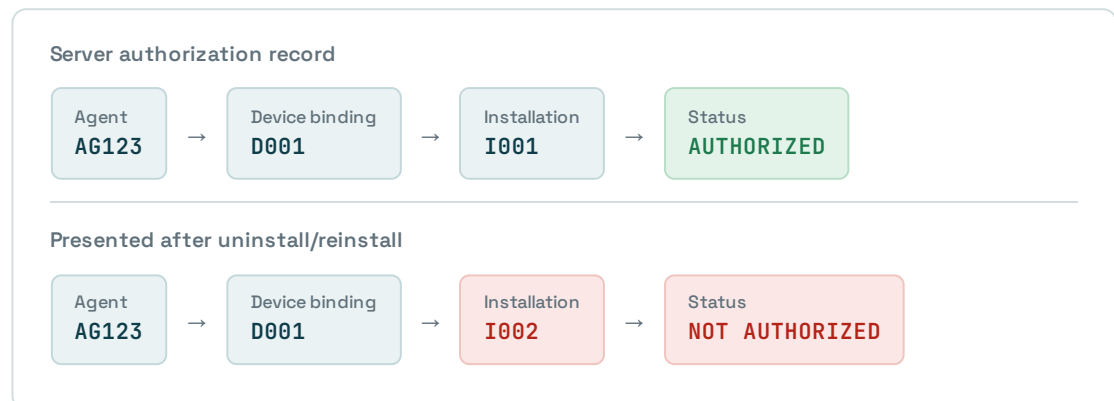


Figure 2 Login decision. Any mismatch leads to a blocked, administrator-resolved state.

- 1 Agent authenticates.
- 2 The server retrieves the current authorized installation and device binding for the agent.
- 3 The server compares the presented installation identity and authenticated device-binding proof against the authorized state.
- 4 If the presented installation/device context matches the authorized state, normal operation continues.
- 5 If it does not match, the server creates a security event and places the agent in **RECONCILIATION_REQUIRED** or **BLOCKED_COLLECTION** state.
- 6 The agent cannot make new collections and cannot authorize the new installation himself.
- 7 The administrator is notified through the configured administrative alert channel.

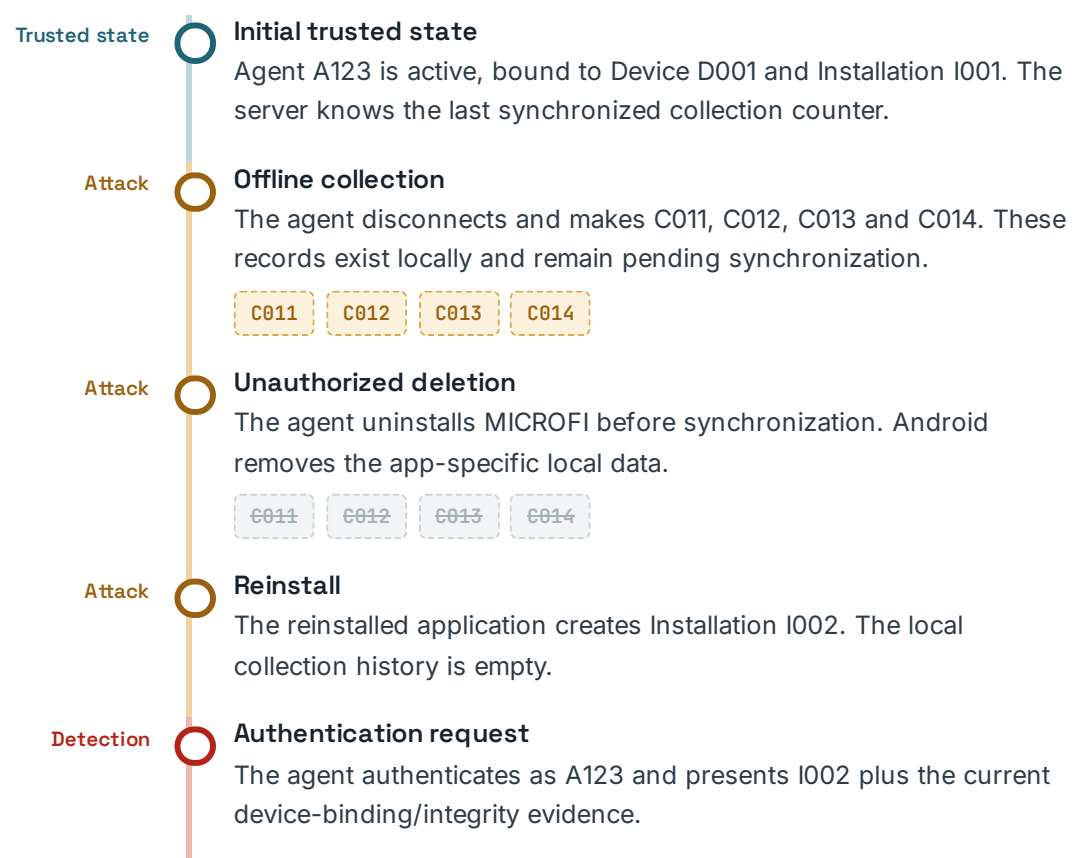
8.2 Why the server can detect the situation

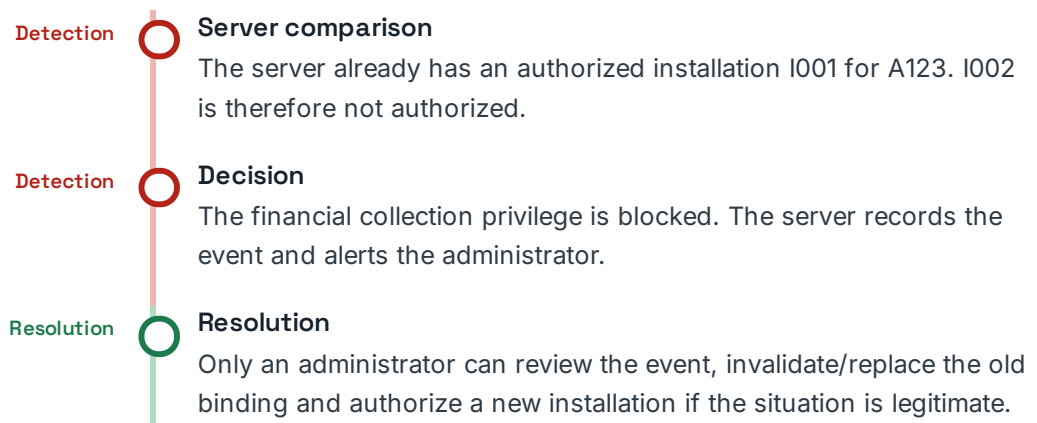
The server stores a persistent authorization record such as:



After uninstall/reinstall, the new app may present I002 instead of I001. The server does not need to know whether the agent uninstalled the app, reset the phone, or performed another local action. Because the policy allows only the currently authorized installation, I002 is simply not authorized.

9 Fraud Scenario: Offline Collections Then Uninstall/Reinstall





10 Administrator-Only Reset / Replacement

A reset is an exceptional privileged operation. It must never be exposed as a self-service operation to the field agent.

- 1 Administrator authenticates with elevated privileges.
- 2 Administrator reviews the security event, agent history, collection checkpoint and device/install history.
- 3 Administrator explicitly selects **Reset / Replace Installation**.
- 4 Server invalidates the prior installation binding.
- 5 Server authorizes one new installation/device binding.
- 6 The operation is recorded in an immutable/auditable administrative log containing administrator identity, time, reason, previous binding and new binding.
- 7 The newly authorized installation repeats the required online onboarding and first-collection procedure before offline collection is re-enabled.

11 Daily Collection Ceiling (Escrow Limit)

Each agent has a daily collection ceiling configured at agent creation. This is a financial risk-containment control, not a cryptographic control.

Definition

For a given business day, the sum of accepted collections for an agent must not exceed the configured daily ceiling.

The limit must be enforced twice:

Enforced locally

So the agent cannot normally exceed the limit while offline.

Enforced server-side

So synchronized/replayed records cannot exceed the limit.

The local implementation must not rely only on a user-changeable wall clock. The application should maintain a trusted server-time baseline and use a monotonic elapsed-time source to detect suspicious clock changes. A configurable maximum offline-duration policy may additionally be imposed by the institution; this specification leaves that value as a business-policy parameter rather than inventing one.

12 Device Binding and Cryptographic Protections

The first version of the design must not assume that an ordinary Android identifier is a permanent hardware identity. The secure binding should be established through cryptographic material and server-verifiable device/app integrity signals.

12.1 Installation key

- ◆ On enrollment, generate an asymmetric signing key in Android Keystore where supported.
- ◆ Register the corresponding public key and binding metadata with the server.
- ◆ Use the private key to sign important collection records and security-sensitive requests.
- ◆ Never store the private key as a plain application file.

12.2 Attestation / integrity

Use a server-verifiable Android integrity mechanism such as Play Integrity for application/device integrity signals, and use Keystore key attestation where the deployment and device support it. These controls help the backend decide whether a request is coming from a genuine, authorized app/device context. They do not replace the server authorization record.

12.3 Why the signature matters

Without a signing key, a technically capable attacker controlling the phone can delete the old ledger and generate a new self-consistent hash chain. Signing makes the record attributable to the protected key associated with the authorized installation/device context, while the server checkpoint provides an external trust anchor.

13 Security States

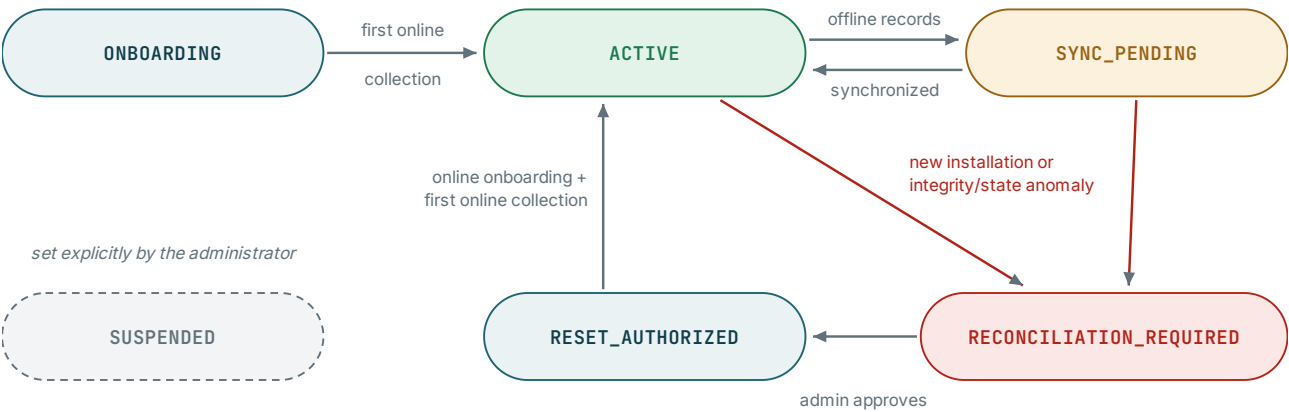


Figure 3 State flow as described in this specification. SUSPENDED is an explicit administrative decision.

State	Meaning	Collection behavior
ONBOARDING	Agent activated; first online collection still required.	Offline collection disabled.
ACTIVE	Authorized installation/device; first online collection completed.	Online and offline collection allowed subject to rules.
SYNC_PENDING	Offline records exist and are not yet fully synchronized.	Offline collection may continue subject to policy/limits.
RECONCILIATION_REQUIRED	New installation or integrity/state anomaly detected.	No new collections; admin review required.
SUSPENDED	Administrator has explicitly suspended the agent.	No collection privileges.
RESET_AUTHORIZED	Administrator has approved a new installation/device binding.	New installation must complete online onboarding before offline mode.

14 Reference Pseudocode

ON_AGENT_ACTIVATION

see section 4

```
ON_AGENT_ACTIVATION(agent):
    agent.status = ONBOARDING
    agent.dailyCeiling = CONFIGURED_LIMIT
```

ON_FIRST_LOGIN

see sections 4 and 12

```
ON_FIRST_LOGIN(agent, installation):
    REQUIRE_ONLINE()
    keyPair = CREATE_KEY_IN_ANDROID_KEYSTORE()
    integrity = REQUEST_DEVICE_APP_INTEGRITY()
    VERIFY_ENROLLMENT(integrity, keyPair.publicKey)
    SAVE_AUTHORIZED_BINDING(agent, installation.id, deviceBinding, keyPair.publicKey)
    ENABLE_OFFLINE = false
```

ON_FIRST_COLLECTION

see sections 4 and 5

```
ON_FIRST_COLLECTION(agent, collection):
    REQUIRE_ONLINE()
    REQUIRE_AUTHORIZED_INSTALLATION()
    REQUIRE_WITHIN_DAILY_CEILING(collection.amount)
    collection.counter = INITIALIZE_COUNTER()
    collection.previousHash = GENESIS_HASH
    collection.currentHash = HASH(CANONICAL_DATA + counter + previousHash)
    collection.signature = SIGN(keyPair.privateKey, canonicalRecord)
    SERVER_ACCEPT(collection)
    SAVE_SERVER_CHECKPOINT(counter, currentHash)
    ENABLE_OFFLINE = true
```

ON_NEW_COLLECTION

see sections 5 and 6

```
ON_NEW_COLLECTION(agent, collection):
    REQUIRE_AUTHORIZED_INSTALLATION()
    REQUIRE_WITHIN_DAILY_CEILING(collection.amount)
    counter = previousCounter + 1
    previousHash = previousCollection.currentHash
    currentHash = HASH(CANONICAL_DATA + counter + previousHash)
    signature = SIGN(keyPair.privateKey, canonicalRecord)
    IF ONLINE:
        SERVER_ACCEPT(collection, counter, previousHash, currentHash, signature)
        UPDATE_SERVER_CHECKPOINT(counter, currentHash)
    ELSE:
        STORE_ENCRYPTED_LEDGER(collection, counter, previousHash, currentHash, signature)
```

ON_LOGIN

see section 8

```

ON_LOGIN(agent, presentedInstallation, presentedDeviceProof):
    authorized = GET_AUTHORIZED_BINDING(agent)
    VERIFY_DEVICE_AND_APP_PROOF(presentedDeviceProof)
    IF presentedInstallation.id ≠ authorized.installationId
        OR presentedDeviceProof does not match authorized binding:
            CREATE_SECURITY_EVENT(agent, presentedInstallation.id)
            SET_AGENT_STATE(agent, RECONCILIATION_REQUIRED)
            BLOCK_COLLECTION(agent)
            NOTIFY_ADMIN(agent)
            RETURN RESTRICTED
    RETURN ALLOWED

```

ON_SYNC

see section 7

```

ON_SYNC(agent, pendingRecords):
    VERIFY_AUTHORIZED_INSTALLATION(agent)
    VERIFY_SIGNATURES(pendingRecords)
    VERIFY_COUNTER_CONTINUITY(pendingRecords)
    VERIFY_HASH_CHAIN(pendingRecords)
    VERIFY_IDEMPOTENCY_AND_DUPLICATES(pendingRecords)
    VERIFY_DAILY_CEILING(pendingRecords)
    ACCEPT_IN_COUNTER_ORDER()
    UPDATE_SERVER_CHECKPOINT()

```

15 Security Boundary and Limitations

- ◆ The server cannot know the exact number or amount of offline collections that were made and then erased before any synchronization event. Information that never reached the server cannot be reconstructed by the server.
- ◆ The hash chain protects the integrity/continuity of the local history but does not by itself detect uninstall. Installation authorization and server-side state provide the reset-control boundary.
- ◆ A daily collection ceiling limits financial exposure per business day. It does not by itself prevent local data loss or prove that an uninstall occurred.
- ◆ The new-installation block is intentionally strict because agent-initiated uninstall, factory reset and device replacement are prohibited by policy. Legitimate replacement is an administrator-controlled exception.
- ◆ Security mechanisms must be validated against the exact Android distribution model, OS versions, rooting/custom-ROM policy, and whether the app is distributed through Google Play or another controlled channel.

16 End-to-End Algorithm Summary



- 1 Activate the agent and configure the daily collection ceiling.
- 2 Perform online onboarding and bind the initial installation/device.
- 3 Force the first collection to be online and establish the first trusted counter/checkpoint.
- 4 Allow subsequent online/offline collections, using a signed hash-chained ledger.
- 5 Enforce the daily ceiling both locally and on the server.
- 6 Synchronize pending offline records in counter order and update the server checkpoint.
- 7 If a new installation or unauthorized device context appears, do not grant collection privileges.
- 8 Create a security event and notify the administrator.
- 9 Only the administrator may reset or replace the binding.
- 10 After administrative reset, require fresh online onboarding/first-collection validation before offline mode is enabled again.

17 Implementation Checklist

- ☐ Server tables for Agent, DeviceBinding, Installation, Collection, ServerCheckpoint and SecurityEvent.
- ☐ Unique constraints preventing more than one active authorized installation per agent unless an explicit replacement workflow is in progress.
- ☐ Cryptographic signing and public-key registration using Android Keystore-supported keys.
- ☐ Server-side verification of Play Integrity and/or Keystore attestation as appropriate.
- ☐ Canonical serialization of collection records before hashing/signing.
- ☐ Monotonic collection counter and strict gap/duplicate handling during synchronization.

- ☐ Encrypted local ledger/database with **PENDING** / **SYNCED** / **REJECTED** states.

- ☐ Daily ceiling calculations using server-anchored business date and anti-clock-tampering checks.

- ☐ Immutable/auditable administrator reset log.

- ☐ Automated tests for reinstall, factory reset, app-data deletion, duplicated counters, gaps, reordered records, modified hashes, invalid signatures and daily-limit bypass attempts.

18 Technical References

The following official Android documentation should be used when implementing the Android-specific security controls:

Android app-specific storage

<https://developer.android.com/training/data-storage/app-specific>

Android security best practices

<https://developer.android.com/privacy-and-security/security-best-practices>

Play Integrity API setup and verdicts

<https://developer.android.com/google/play/integrity/setup>

Android Keystore attested key pair reference

<https://developer.android.com/reference/kotlin/android/security/keystore/AttestedKeyPair>
